BellaStyle Manager

Sistema de gestión para salón de belleza

Manual Técnico

Arquitectura, base de datos, código fuente e instalación

César Pestana
Proyecto de Aula · Ingeniería de Sistemas
Fundación Universitaria Colombo Internacional
Cartagena de Indias, octubre de 2026 · Versión 3.0

Contenido

Introducción

2. Manual técnico

Glosario

Control del documento

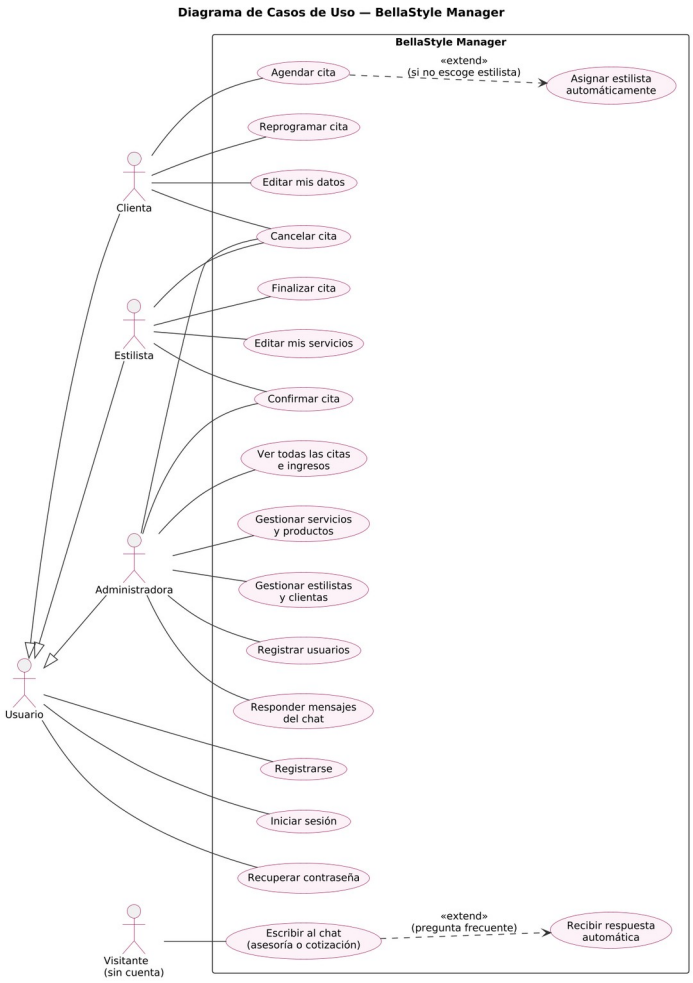
Introducción

Este documento es el **Manual Técnico** de **BellaStyle Manager**. Va dirigido al personal de soporte técnico o a quien haga sus veces, y detalla la estructura del sistema para entender su funcionamiento y facilitar su instalación y mantenimiento.

Sigue la estructura de la guía *Manuales técnicos de la aplicación* (SENA): alcance, descripción de los procesos, diagramas UML, administración de usuarios, modelo relacional de la base de datos, descripción de la plataforma, documentación del código fuente y acuerdos de niveles de servicio (ANS).

2. Manual técnico

No	Descripción	Detalle
1	Alcance	**Objetivo general:** desarrollar un sistema web que permita al salón BellaStyle gestionar sus citas, su personal y su catálogo de servicios y productos. **Objetivos específicos cumplidos:** • Registro e inicio de sesión con tres roles: Administradora, Estilista y Clienta, con contraseñas cifradas. • Agendamiento de citas con reglas de horario y **asignación automática** de la estilista según servicio y carga de trabajo. • Reprogramación y cambio de estado de las citas (Pendiente, Confirmada, Finalizada, Cancelada). • Gestión de servicios, productos, estilistas y clientas por parte de la administradora. • Indicadores de ingresos (administradora) y de comisión (estilista). • **Chat de asesoría** en la página de inicio: el visitante escribe sin cuenta, un asistente automático responde las preguntas frecuentes y la administradora contesta desde su panel. **Límites (no cubiertos):** pagos y facturación, proveedores, movimientos de inventario, reportes impresos y notificaciones por correo.
2	Descripción de los procesos	El sistema apoya el proceso principal del salón, **gestionar citas**, y los procesos de apoyo de **gestión de usuarios** y **gestión del catálogo**. El diagrama de casos de uso muestra qué hace cada actor:

No	Descripción	Detalle
		 *Figura 1. Diagrama de casos de uso* **Requerimientos funcionales:** **Código** / **Requerimiento** RF01 — Registrar cuentas de clienta, estilista y administradora (con código). RF02 — Iniciar sesión por rol y recuperar la contraseña. RF03 — Agendar citas validando horario, domingos, fechas pasadas y cruces. RF04 — Asignar automáticamente la estilista libre con menos citas del día. RF05 — Reprogramar y cancelar citas (clienta). RF06 — Confirmar y finalizar citas (estilista y administradora). RF07 — Crear, editar, activar y desactivar servicios y productos. RF08 — Activar o desactivar estilistas y asignarles servicios.

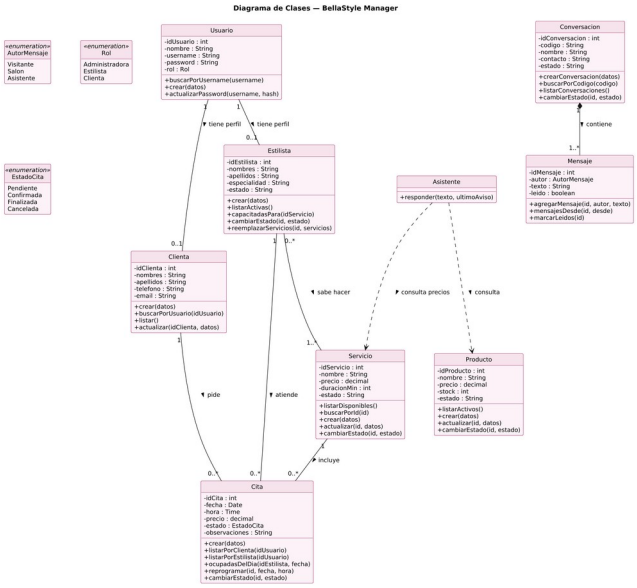
<table>
<tr><th>No</th><th>Descripción</th><th>Detalle</th></tr>
<tr><td></td><td></td><td>
<table>
<tr><td>RF09</td><td>Editar datos de clientas.</td></tr>
<tr><td>RF10</td><td>Mostrar ingresos y comisión (40 %) de las citas finalizadas.</td></tr>
<tr><td>RF11</td><td>Permitir que un visitante sin cuenta escriba al salón por un chat en la página de inicio.</td></tr>
<tr><td>RF12</td><td>Responder automáticamente preguntas frecuentes (precios, horario, servicios, productos, cómo agendar).</td></tr>
<tr><td>RF13</td><td>Notificar mensajes nuevos (contadores) y permitir que la administradora responda y cierre conversaciones.</td></tr>
</table>
</td></tr>
<tr><td>3</td><td>Diagramas UML</td><td>Diagrama de clases. Representa las entidades del sistema, sus atributos, las operaciones de cada modelo y sus relaciones:
 

Figura 2. Diagrama de clases

Diagrama de secuencia – Agendar cita. Muestra cómo viaja la petición por las capas hasta la base de datos:</td></tr>
</table>

No	Descripción	Detalle

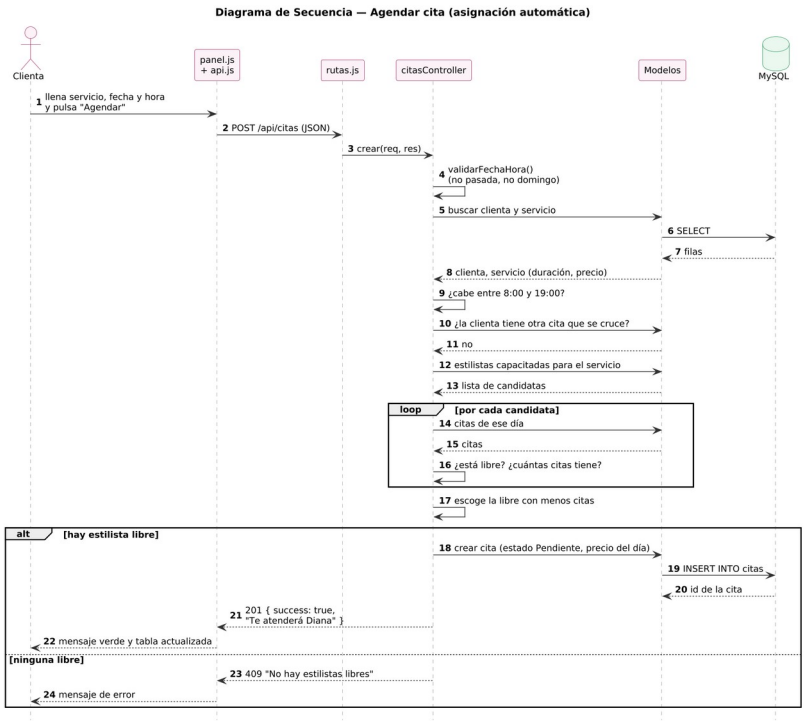


Figura 3. Diagrama de secuencia: agendar cita

Diagrama de secuencia – Chat de asesoría. Muestra cómo el mensaje del visitante llega a la administradora y cómo vuelve la respuesta. Como el chat usa *polling*, cada navegador pregunta cada pocos segundos si hay mensajes nuevos:

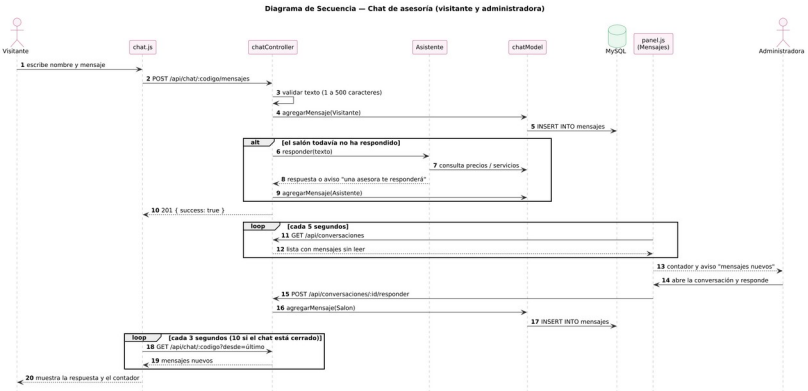


Figura 3b. Diagrama de secuencia: chat de asesoría

Diagrama de componentes. Los componentes se conectan por **interfaces**: la bola es la interfaz que un componente **provee** y la media luna la que **requiere**.

No	Descripción	Detalle

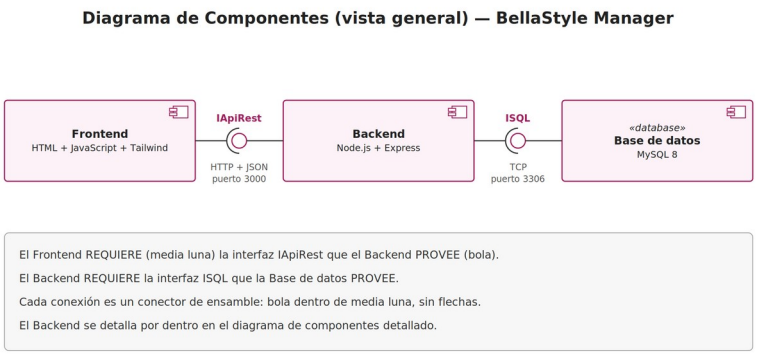


Figura 4. Componentes: vista general

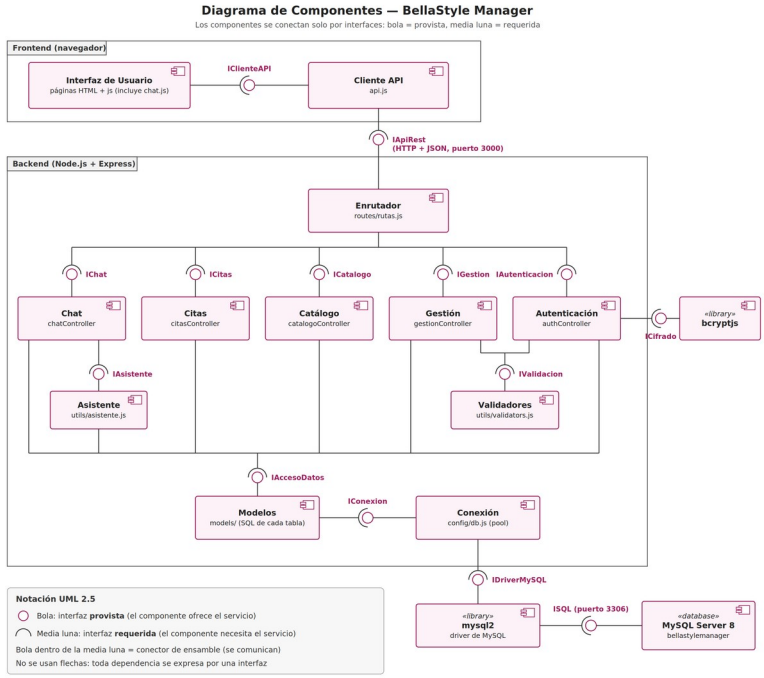


Figura 5. Componentes: vista detallada

Diagrama de despliegue. Muestra dónde se ejecuta cada parte: navegador, servidor Node.js y servidor MySQL, comunicados por HTTP (puerto 3000) y TCP (puerto 3306).

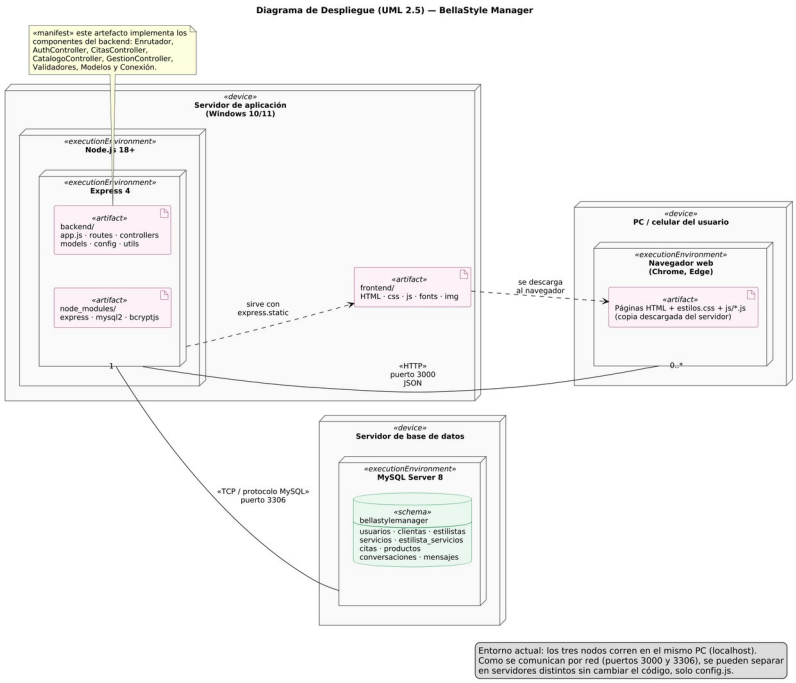
<table>
<tr><th>No</th><th>Descripción</th><th>Detalle</th></tr>
<tr><td></td><td></td><td> 

Figura 6. Diagrama de despliegue</td></tr>
<tr><td>4</td><td>Administración de usuarios</td><td>a. Creación, modificación y retiro de usuarios:
Creación: cada persona se registra en <code>register.html</code>, o la administradora la crea en la pestaña Registrar usuario. La cuenta (<code>usuarios</code>) y su perfil (<code>clientas</code> o <code>estilistas</code>) se guardan en una transacción.
Modificación: la administradora edita los datos de las clientas y los servicios de las estilistas; la clienta edita sus propios datos; cualquier usuario cambia su contraseña con ¿La olvidaste?.
Retiro: las estilistas se desactivan (no reciben citas nuevas) en lugar de borrarse, para conservar el historial.
b. Roles y perfiles. Los roles están definidos en la columna <code>usuarios.rol</code>:<table><tr><th>Rol</th><th>Permisos</th></tr><tr><td>Clienta</td><td>Agendar, reprogramar y cancelar sus citas; editar sus datos.</td></tr><tr><td>Estilista</td><td>Ver sus citas; confirmar, finalizar o cancelar; editar sus servicios.</td></tr><tr><td>Administradora</td><td>Ver todas las citas e ingresos, confirmarlas o cancelarlas; gestionar servicios, productos, estilistas, clientas y cuentas. Se registra con el código de <code>config.js</code>.</td></tr><tr><td>Visitante</td><td>No tiene cuenta. Solo puede escribir en el chat de la página de inicio; se reconoce por el código guardado en</td></tr></table></td></tr>
</table>

No	Descripción	Detalle
		su navegador.
5	Modelo relacional de la base de datos	Base `bellastylemanager` en MySQL 8 (`utf8mb4`), creada con `init_db.sql`:

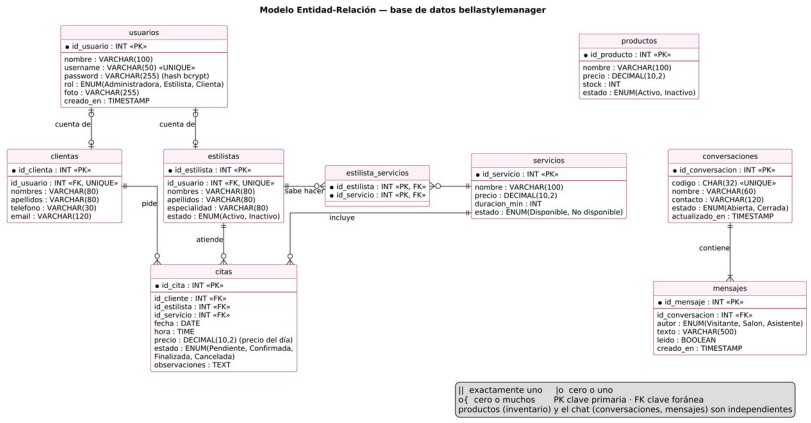


Figura 7. Modelo relacional

Diccionario de datos:

Tabla	Columna	Tipo	Descripción
usuarios	id_usuario	INT PK	Identificador de la cuenta.
	username	VARCHAR(50) UNIQUE	Usuario para iniciar sesión.
	password	VARCHAR(255)	Hash bcrypt de la contraseña.
	rol	ENUM	Administradora, Estilista o Clienta.
clientas	id_clienta	INT PK	Identificador de la clienta.
	id_usuario	INT FK UNIQUE	Cuenta asociada (puede ser nula).
	nombres, apellidos, telefono, email	VARCHAR	Datos de contacto.
estilistas	id_estilista	INT PK	Identificador de la estilista.
	id_usuario	INT FK UNIQUE	Cuenta asociada.
	estado	ENUM	Activo o Inactivo.
servicios	id_servicio	INT PK	Identificador del servicio.

No	Descripción	Detalle			
			precio / duracion_min	DECIMAL / INT	Precio y duración en minutos.
			estado	ENUM	Disponible o No disponible.
		estilista_servicios	id_estilista, id_servicio	INT PK FK	Qué servicios sabe hacer cada estilista (N a M).
		citas	id_cita	INT PK	Identificador de la cita.
			id_cliente, id_estilista, id_servicio	INT FK	Clienta, estilista y servicio.
			fecha / hora	DATE / TIME	Inicio de la cita.
			precio	DECIMAL(10,2)	Precio del día en que se agendó.
			estado	ENUM	Pendiente, Confirmada, Finalizada o Cancelada.
		productos	id_producto	INT PK	Identificador del producto.
			precio / stock / estado	DECIMAL / INT / ENUM	Precio, unidades y Activo/Inactivo.
		conversaciones	id_conversacion	INT PK	Identificador de la conversación del chat.
			codigo	CHAR(32) UNIQUE	Código aleatorio que el navegador del visitante guarda para reconocerlo sin cuenta.
			nombre / contacto	VARCHAR	Nombre y teléfono o correo (opcional).
			estado	ENUM	Abierta o Cerrada.
		mensajes	id_mensaje	INT PK	Identificador del mensaje.
			id_conversacion	INT FK	Conversación a la que pertenece (ON DELETE CASCADE).
			autor	ENUM	Visitante, Salon o Asistente.
			texto / leido	VARCHAR(500) / BOOLEAN	Contenido y si la administradora ya lo

<table>
<tr><th>No</th><th>Descripción</th><th>Detalle</th></tr>
<tr><td></td><td></td><td>
<table>
<tr><td></td><td></td><td></td><td>vio.</td></tr>
</table>
</td></tr>
<tr><td>6</td><td>Descripción de la plataforma</td><td>
BellaStyle Manager es una aplicación web cliente-servidor. El servidor está hecho en Node.js con Express y la base de datos es MySQL. La interfaz está desarrollada en HTML5, JavaScript y Tailwind CSS.
<table>
<tr><th>Herramienta</th><th>Versión</th><th>Uso</th></tr>
<tr><td>Node.js</td><td>18 o superior</td><td>Entorno de ejecución del servidor.</td></tr>
<tr><td>Express</td><td>4.x</td><td>Rutas de la API y archivos estáticos.</td></tr>
<tr><td>MySQL Server</td><td>8.x</td><td>Sistema manejador de base de datos (SGBD).</td></tr>
<tr><td>mysql2</td><td>3.x</td><td>Librería de conexión a MySQL (pool).</td></tr>
<tr><td>bcryptjs</td><td>2.4</td><td>Librería de cifrado de contraseñas.</td></tr>
<tr><td>Tailwind CSS</td><td>3.4</td><td>Estilos, compilados con su CLI.</td></tr>
</table>
Para instalar el servidor se requiere:
1. Windows 10/11 (también funciona en Linux o macOS), 4 GB de RAM y 1 GB libre en disco.
2. Node.js 18 o superior y MySQL Server 8.
3. Ejecutar <code>INSTALAR_BASE_DE_DATOS.bat</code> (crea la base y los datos de prueba).
4. Revisar usuario y contraseña de MySQL en <code>backend/config/config.js</code>.
5. Ejecutar <code>backend/INICIAR_SERVIDOR.bat</code> y abrir <code>http://localhost:3000</code>.
Para el mantenimiento se requiere: Visual Studio Code (o similar), MySQL Workbench y npm. Si se cambian clases de Tailwind, ejecutar <code>npm run css</code> en la carpeta <code>backend</code>.
</td></tr>
<tr><td>7</td><td>Documentación del código fuente</td><td>
1. Arquitectura. El backend está organizado en capas: rutas → controladores → modelos → base de datos. Las rutas reciben la petición, los controladores aplican las reglas de negocio y solo los modelos ejecutan SQL.
2. Estructura de directorios:
<table>
<tr><th>Directorio</th><th>Descripción</th></tr>
<tr><td>backend/</td><td>Servidor Node.js. <code>app.js</code> es el arrancador de la aplicación.</td></tr>
<tr><td>backend/config</td><td>Configuración (puerto, MySQL, código admin) y pool de conexiones.</td></tr>
<tr><td>backend/routes</td><td>Lista de rutas (endpoints) de la API.</td></tr>
<tr><td>backend/controllers</td><td>Lógica y reglas de negocio.</td></tr>
</table>
</td></tr>
</table>

No	Descripción	Detalle

backend/models	Consultas SQL de cada tabla.
backend/utils	Validaciones reutilizables y asistente automático del chat.
frontend/	Páginas HTML que ve el usuario.
frontend/js	Lógica del navegador y llamadas a la API.
frontend/src y css	Estilos de Tailwind: fuente y archivo compilado.
frontend/img y fonts	Imágenes y fuentes locales.

3. Relación de programas:

Programa	Lenguaje	Descripción
backend/app.js	JavaScript	Arranca Express, sirve el frontend y monta la API en /api.
config/config.js · db.js	JavaScript	Datos de conexión y pool de MySQL.
routes/rutas.js	JavaScript	Asocia cada URL con su controlador.
controllers/authController.js	JavaScript	Login, registro y recuperar contraseña.
controllers/citasController.js	JavaScript	Citas: reglas de horario y asignación automática.
controllers/ catalogoController.js	JavaScript	Consultas de servicios, productos, estilistas y clientas.
controllers/ gestionController.js	JavaScript	Crear, editar, activar y desactivar.
models/*.js (6 archivos)	JavaScript / SQL	Una clase de acceso a datos por tabla.
utils/validators.js	JavaScript	Validación de nombre, usuario y contraseña.
controllers/chatController.js	JavaScript	Chat: iniciar conversación, enviar, leer, responder y cerrar.
models/chatModel.js	JavaScript / SQL	Consultas de las tablas conversaciones y mensajes.
utils/asistente.js	JavaScript	Respuestas automáticas por palabras clave.
instalar_bd.js	JavaScript	Ejecuta init_db.sql.
init_db.sql	SQL	Crea las tablas y los datos de prueba.
welcome · index · register ·	HTML	Pantallas de la aplicación.

No	Descripción	Detalle

Programa	Lenguaje	Descripción
recuperar · panel (.html)		
js/api.js	JavaScript	Función `pedir()` (fetch) y manejo de sesión.
js/login.js · registro.js · inicio.js	JavaScript	Lógica de inicio de sesión, registro y página de inicio.
js/panel.js	JavaScript	Paneles de los tres roles (incluye la pestaña Mensajes).
js/chat.js	JavaScript	Burbuja del chat en la página de inicio.
src/estilos.css	CSS (Tailwind)	Componentes de estilo reutilizables.

4. API REST. Todas las rutas empiezan por `/api` y responden JSON `{ success, message, data }`:

Método	Ruta	Descripción
POST	/login · /register · /recuperar-password	Cuentas.
GET / POST	/citas	Listar y crear citas.
PUT	/citas/:id/estado · /citas/:id/reprogramar	Cambiar estado o fecha.
GET	/servicios · /productos · /estilistas · /clientas	Consultas.
POST / PUT	/servicios · /productos · /clientas/:id	Crear y editar.
PUT	/.../:id/estado · /estilistas/:id/servicios	Activar, desactivar, asignar.
POST	/chat	Inicia una conversación (visitante).
GET / POST	/chat/:codigo · /chat/:codigo/mensajes	Leer mensajes nuevos y enviar (visitante).
GET	/conversaciones · /conversaciones/:id/mensajes	Lista y lectura (administradora).
POST / PUT	/conversaciones/:id/responder · /:id/estado	Responder, cerrar o reabrir.

5. Regla principal: asignación automática. De las estilistas activas que hacen el servicio, se descartan las que tienen una cita que se cruza y se escoge la que tiene menos citas ese día. Dos horarios se cruzan así:

```
function seCruzan(inicioA, finA, inicioB, finB) {
    return inicioA < finB && inicioB < finA;
```

<table>
<tr><th>No</th><th>Descripción</th><th>Detalle</th></tr>
<tr><td></td><td></td><td>

```
}
```

6. Chat y asistente automático. El visitante no inicia sesión: al empezar el chat el servidor genera un código aleatorio de 32 caracteres (`crypto.randomBytes`) que el navegador guarda en `localStorage`. El asistente (`utils/asistente.js`) normaliza el texto (minúsculas y sin tildes) y busca palabras clave; por ejemplo, si encuentra *precio* consulta la tabla `servicios` y responde la lista real. Solo responde mientras el salón no haya escrito en esa conversación.

7. Seguridad: contraseñas con bcrypt, consultas parametrizadas (evitan inyección SQL), registro en transacción, validación en el servidor y textos del chat escapados antes de mostrarlos (evita XSS).
</td></tr>
<tr><td>8</td><td>Acuerdos de niveles de servicio (ANS)</td><td>
Los casos se reciben en la mesa de ayuda y se atienden según su prioridad:
<table>
<tr><th>Prioridad</th><th>Ejemplo</th><th>Respuesta</th><th>Solución</th></tr>
<tr><td>Alta</td><td>El servidor no inicia o no hay conexión con MySQL; nadie puede ingresar.</td><td>1 hora</td><td>4 horas</td></tr>
<tr><td>Media</td><td>Una función falla (no se puede agendar, editar o cambiar estados).</td><td>4 horas</td><td>1 día hábil</td></tr>
<tr><td>Baja</td><td>Dudas de uso, cambios de datos, textos o mejoras.</td><td>1 día hábil</td><td>3 días hábiles</td></tr>
</table>
Escalamiento: el primer nivel (administradora del salón) resuelve cuentas, servicios, precios y datos. Si no puede resolverlo, escala al segundo nivel (soporte técnico), que revisa el servidor, la base de datos y el código.
</td></tr>
</table>

Glosario

ANS: acuerdo de niveles de servicio. Define tiempos de respuesta y escalamiento de los casos de soporte.

API REST: conjunto de direcciones (rutas) por las que el navegador pide y envía datos al servidor en formato JSON.

bcrypt: algoritmo que cifra las contraseñas de forma que no se puedan leer.

Librería: conjunto de programas que apoyan un área específica y que el programador reutiliza (por ejemplo, mysql2).

Polling: técnica en la que el navegador pregunta al servidor cada pocos segundos si hay datos nuevos. La usa el chat.

Pool de conexiones: grupo de conexiones a la base de datos que se reutilizan para responder más rápido.

SGBD: sistema manejador de bases de datos; en este proyecto, MySQL.

UML: lenguaje unificado de modelado, usado para diseñar sistemas con diagramas.

Control del documento

Campo	Detalle
Documento	Manual Técnico – BellaStyle Manager
Proyecto	Proyecto de Aula · Ingeniería de Sistemas
Institución	Fundación Universitaria Colombo Internacional
Autor	César Pestana
Versión	3.1 – octubre de 2026 (estructura guía SENA; chat y nuevo inicio de sesión)
Referencia	SENA. Manuales técnicos de la aplicación. IEEE 1063-2001, Standard for Software User Documentation.